Training in “Realistic” Environments

The route that skips the simulator, and what the quotation marks are doing

---

The page states the route in one line, before anything is argued for.

> *“If transfer from a simulator is where the fidelity is lost, train where the fidelity is.”*
>
> website, training-in-realistic-environments.md, opening

That sentence is a bet, and this file is the vocabulary the bet is made in. Three kinds of word are in it. Words for the route itself, which is the choice not to have a simulator stage. Words for the defender’s action, which is deception and not blocking or patching. And words for the environment, which is a provisioned cloud network with a command-and-control server standing in for the usual transition function.

The method that runs on this route has its own file. `cwm_foe_dreamer.tex` holds the factored world model, the opponent embedding, the metrics and the reviewers. What is here is what the route costs, what it buys, and the one word in the title that is doing the most work while wearing quotation marks.

Contents

1 Active

1.1 The route

the simulator-and-transfer route

"Most autonomous cyber defense trains in a simulator and transfers the resulting policy."

website, training-in-realistic-environments.md

Definition: The default pipeline in autonomous cyber defense. Fit a policy in a cheap abstract environment, then move it to the network it is meant to protect, and treat the move as an engineering step rather than as part of the result.

Distinguishes it from: The route this project takes, which has one environment and no move. And from sim2sim, which is the same move studied on purpose rather than performed and assumed to work, and which has its own file.

Operational test: Count the environment implementations a policy sees before it is evaluated. Two is this route. One is the other.

In my project: Named in the second section of the page as what the field usually does, immediately before what it costs.

what the simulator omits

"Simulators omit authentication delays, partial action failures, service dependencies, and realistic user traffic."

website, training-in-realistic-environments.md

Definition: Four named properties, not a general complaint about abstraction. Each is a thing a defender's best action depends on, and each is absent from the environments the field trains in.

Distinguishes it from: A fidelity score. This is an inventory of specific omissions, so it can be checked one item at a time, and a given environment can be credited for the ones it does carry.

Operational test: For each of the four, ask whether the training environment represents it at all. Absent means the policy never had to handle it and the evaluation never charged it for failing to.

In my project: The four are the reason the page treats the fidelity loss as located at the transfer step rather than spread evenly through the pipeline.

the 66 percent figure

"Simulator-trained policies succeed only 66% of the time on the matching emulator, and realistic training environments and sim-to-real transfer remain the field's two principal open problems."

website, training-in-realistic-environments.md

Definition: The one number the page uses to price the transfer step: a policy that works in the simulator works two times in three on the emulator built to match it. The emulator is the friendly case, since it was constructed to correspond to that simulator.

Distinguishes it from: A generalization gap across attackers or topologies. Nothing about the task changed here. Only the implementation did.

Operational test: Was the target built to match the source? If yes, the number is a floor on the loss and not a worst case.

In my project: It is the evidence behind the fidelity gap in the manuscript, Section 2.2, and the reason the page can say the field has two open problems rather than one.

no simulator stage

"it trains the defender directly in an emulated operational network, with no simulator stage"

website, training-in-realistic-environments.md

Definition: Every real transition the learner is fitted on comes from the emulation. There is no second environment implementation between the network and the policy.

Distinguishes it from: The two-backend arrangement described below, which is a property of the testbed and not of the training run. The simulator backend exists. The claim is that the reported policy was not fitted on it.

Operational test: Ask which backend produced the transitions in the replay buffer. If the answer is the live one, the claim holds whatever else the testbed can also do.

In my project: Stated in the manuscript abstract and carried onto the page. The full entry, including the reviewer who reads the learned world model as a simulator by another name, is in `cwm_foe_dreamer.tex`.

whether the route pays

"whether the route pays"

website, training-in-realistic-environments.md

Definition: The question the page is organized around. Not whether training in place is more realistic, which is settled by construction, but whether the result is reachable inside a budget somebody would actually spend.

Distinguishes it from: An architecture claim. The page hands the architecture to another page and keeps the economics.

Operational test: Name the budget and the hardware before naming the score. If the sentence cannot be completed with both, it is not answering this question.

In my project: The whole of the experiment section reduces to it, and the answer is three days on one GPU.

1.2 Deception as the defender's action

deception policy

"A reinforcement-learning defender whose actions are deception"

artifacts/daedalus/README.md, opening

Definition: A defender whose action space contains no removals and no repairs. Every action stands something up: a decoy service, a false privilege path, planted data. The target of the action is the attacker's belief rather than the attacker's access.

Distinguishes it from: Containment and remediation policies, whose actions are isolate, patch, restore. Those change the network. A deception action changes what the network appears to be, and leaves the real access it is hiding intact.

Operational test: After the action fires, is the attacker's actual position different? If only the attacker's picture of it changed, the action was deception.

In my project: The Daedalus action set. The page for the method describes the same policy in defensive terms, and this is the file that says what the actions are.

shaping what the attacker believes

"It is to shape what the attacker believes: stand up a decoy service, fake a privilege-escalation path, plant bait data"

artifacts/daedalus/README.md, The idea

Definition: The objective of a deception action, stated as a target rather than as an effect. Success is that the attacker spends its next move on something that is not real, and reveals itself by spending it.

Distinguishes it from: Denial. Denial removes an option and the attacker learns that immediately. Deception adds a false option, and the attacker learns nothing until it has already committed to it.

Operational test: Did the attacker act on the thing that was placed? A decoy nobody touches has cost the defender and taught it nothing.

In my project: This is why the observation carries the attacker's perceived state. A policy optimizing a belief has to be able to see the belief.

fake node

Definition: A Cowrie honeypot deployed on a real host, presenting a service the attacker can reach and interact with, and which exists to be reached.

Distinguishes it from: A decoy host in a simulator, which is a flag in a state vector and has no service behind it. Here the honeypot is a real process on a real operating system and the attacker meets whatever that process presents.

Operational test: Could the attacker connect to it and receive a response generated by software rather than by a transition function? Only then is it this.

In my project: One of the four defensive actions, toggled through the C2 server.

fake edge

Definition: A SUID binary placed so that the host appears to offer a privilege-escalation path it does not really offer. The word edge is the graph term: the defender is adding an edge to the attack graph the attacker believes it is on.

Distinguishes it from: The fake node, which adds a vertex. This adds a transition between

two hosts or two privilege levels the attacker already knows about, which is a cheaper lie and a more specific one.

Operational test: Does the placed object change what the attacker thinks it can reach from where it already is? That is an edge. Does it change what exists at all? That is a node.

In my project: One of the four defensive actions. Named in the artifact pages as the SUID fake edge, and toggled on a real host like the other three.

bait data

Definition: Planted data on a real host, placed so that an attacker looking for something worth taking finds it and takes it.

Distinguishes it from: The other two placements, which alter the attacker's route. This one alters the attacker's objective, and it is the only one of the three that pays off after the attacker has already succeeded at something.

Operational test: Would the attacker's own success condition be satisfied by the placed object? If yes, it is bait and not a decoy.

In my project: The third of the four actions, called fake data in the design page and the fake-data bait in the use page.

do nothing

"Deploy a Cowrie honeypot (a fake node), deploy a SUID fake edge (a fake privilege path), plant fake data, or do nothing. Each is a real service the C2 server toggles on a real host."

artifacts/daedalus/README.md, The pieces

Definition: The fourth action. A deliberate no-op, present because every placement costs something and because a defender that must place on every step is not being asked the question the environment is built to ask.

Distinguishes it from: An action mask. The policy may take this action when others are legal, so choosing it is a decision and gets scored as one.

Operational test: Is there a step at which the best move is to leave the board alone? If the action set cannot express that, the timing half of the problem has been removed from it.

In my project: The use page says the policy learns where to place deception and when. This action is the entire when.

manipulated value

"Each host carries an IP, a value, a manipulated value the defender can shift, and per-host deception state"

artifacts/daedalus/design.md, The network

Definition: The value of a host as the defender has arranged for it to appear, held separately from the value it really has. The defender can move the first and not the second.

Distinguishes it from: The host's value, which is fixed by the scenario and is what the attacker is actually trying to reach. Keeping the two as separate fields is what lets the environment score a lie as a lie rather than as a change to the network.

Operational test: Does the quantity change when the defender acts, while the underlying asset does not? Then it is the manipulated value.

In my project: Per node, and it is the first field the translation layer writes into the observation.

perceived state

"the attacker's perceived state of that node, which is the quantity the whole design is trying to bend away from the truth"

artifacts/daedalus/design.md, The network

Definition: The attacker's model of a node, carried as environment state. The defender does not observe the attacker's reasoning. It observes the standing result of that reasoning, per node, and acts on it.

Distinguishes it from: The node state, which is what is true. The gap between the two is the defender's product, and a deception policy that never opens a gap has done nothing whatever its score says.

Operational test: If the defender takes no action for an episode, does this quantity converge on the node state? It should, and how fast it does is a property of the attacker rather than of the defender.

In my project: Section of the design page on the observation, and the last field per node in the vector the agent reads.

1.3 The environment interface

Daedalus

"Daedalus runs the defender against a provisioned OpenStack network with real services, so the deception it deploys is a real honeypot on a real host rather than a flag in a state vector."

artifacts/daedalus/README.md, The idea

Definition: The eight-host OpenStack environment across three subnets, its scripted red agent, its generated background traffic, its command-and-control server, and the two backends behind one interface. The environment, not the learner.

Distinguishes it from: FOE-Dreamer, which is the method trained and evaluated in it. The research page names the network and hands the architecture to the method's own page, and this file keeps to the network.

Operational test: Does the sentence describe something that would still exist if the learner were replaced by a random policy? If yes, it belongs to Daedalus.

In my project: Three artifact pages carry it: README for the idea and the pieces, design for the interface, use for the workflow. The research page carries one paragraph of it.

> **Two names for one repository**
>
> The use page opens its training block with `cd CyberDreamer`. The preprint, the research page and the artifact pages all say FOE-Dreamer, and the environment is Daedalus.

> Three names are in play for two things, and the directory carries the older one. Nothing turns on it technically. It matters the first time somebody reads the artifact pages and the paper on the same afternoon, which is the situation this lexicon exists for.

command-and-control as the environment interface

"The agent never touches a host directly."

artifacts/daedalus/design.md, opening

"Each host in the network is a gRPC client; the C2 server holds the connections and decides which host a given action applies to."

artifacts/daedalus/design.md

Definition: The C2 server occupies the position a transition function occupies in a simulator. It receives an action, resolves it to a machine, and performs it there. What comes back is what the machine did.

Distinguishes it from: A step function. A step function computes the consequence of an action from a model of the world. This one causes the consequence and then reports it, and the difference is that it can be wrong about nothing and can also fail.

Operational test: Is there code anywhere that decides what a honeypot does? If the answer is no, because Cowrie is running, the interface is this one.

In my project: The design page, and the reason the same page puts the word environment in quotation marks.

the (node, action) pair

"Deploying a honeypot on the WEB host is action `(2, deploy_honeypot)`; the server resolves index 2 to that machine and enables the real service."

artifacts/daedalus/design.md

Definition: The action format. A node index and a defensive action, which means the policy chooses a placement and a target in one emission rather than selecting from a flat list of host-action combinations.

Distinguishes it from: A flat action space, which grows with the product of hosts and actions. This factors it, and the factoring is what makes the same policy shape run against a network of a different size.

Operational test: Can the action be named without naming a host? If not, it is this pair and the index half is meaningful.

In my project: The simulator implements the same pair, which is what makes the two backends interchangeable at all.

two backends behind one interface

"Two backends sit behind one action and observation interface: a fast simulator for the millions of steps reinforcement learning requires, and the live emulated deployment for evaluation."

website, training-in-realistic-environments.md

Definition: One action space and one observation space, implemented twice. Once cheaply and once honestly. Which backend is attached is a runtime choice and not a change to the agent.

Distinguishes it from: A sim-to-real pipeline, where the two implementations have different interfaces and a translator has to be built between them. Here there is nothing to translate, which is what makes the difference between the two readable as a measurement.

Operational test: Would the agent code notice which backend it is attached to? If it has to be edited, these are not two backends behind one interface.

In my project: The design page and the research page both state it. The policy reported in the manuscript was fitted on the live one.

the honest end

"The OpenStack deployment is the honest end"

artifacts/daedalus/design.md, Two backends behind one interface

Definition: Her name for the live backend, given with its price attached: slow, expensive, and real. The phrase concedes that the other end is cheap and implies what the trade is.

Distinguishes it from: A validation set. A validation set is a held-out sample of the same distribution. This is a different implementation of the same interface, and what it is held out from is the modelling itself.

Operational test: Does a step cost wall-clock seconds and change a running machine? That is the honest end, and the number of steps available there is the budget the whole project is written against.

In my project: The evaluation backend, and the one the reported training run also used.

the fidelity gap made concrete

"the actions that worked in the simulator and failed against the real service stack are the fidelity gap made concrete, which is the point of running on real infrastructure at all"

artifacts/daedalus/use.md

"the difference between the two is itself a measurement of what the simulator abstracted away"

website, training-in-realistic-environments.md

Definition: The per-action disagreement between the two backends, read as data. Not the score gap in aggregate, but the list of actions whose outcome differs, which names the omission rather than sizing it.

Distinguishes it from: The fidelity gap as the manuscript uses it, which is the general obstacle between training and deployment and is defined in `cwm_foe_dreamer.tex`. This is that gap instantiated as a diff, on one testbed, action by action.

Operational test: Can you name the action and the service it failed against? If the answer is a percentage, the gap has been sized and not made concrete.

In my project: The use page says this is what evaluation reads off, alongside whether the defence held. It is the second output of every evaluation run.

the scripted red agent

"The attacker is a scripted red agent that executes a genuine web exploit, establishes persistence over SSH, and pivots toward the interior hosts, so a defender is measured against real tool execution."

website, training-in-realistic-environments.md

"Because the attack is genuine tool execution rather than a transition function, a defence that looks good against it is being tested against the thing it will actually face."

artifacts/daedalus/design.md, The attacker

Definition: An attacker whose policy is fixed and whose execution is real. The decision rule is scripted. The WordPress exploit, the SSH key generation, the pivot inward are performed against the running hosts.

Distinguishes it from: A learned or adaptive adversary, which this is not, and which the method page keeps open as the next thing. And from a simulated attacker, whose actions succeed because a transition function says so.

Operational test: Split the attacker in two. Is the choice of next action computed, and is the action itself executed? Scripted choice with real execution is this.

In my project: Compiled to a standalone binary. Two fixed-class profiles are used in the evaluation, and those belong to `cwm_foe_dreamer.tex`.

generated background activity

"Background user activity is generated rather than assumed away."

website, training-in-realistic-environments.md

Definition: Ordinary user traffic produced by scripted personas on the same hosts, so that the defender's observation stream contains legitimate activity it has to see past.

Distinguishes it from: A clean baseline, which is the failure the Metrion page describes: a benchmark where attacks raise alerts and ordinary enterprise life is thin, so a defender scores well by reading malice against silence.

Operational test: Remove the attacker for one episode. Is the observation stream empty? An empty stream means the defender was never asked to separate anything.

In my project: GHOSTS personas. The timing structure and the two evaluation profiles are in `cwm_foe_dreamer.tex` under behaviorally correlated user activity.

1.4 What "realistic" is doing

"realistic", with the quotation marks

"An environment is not realistic or unrealistic; it is realistic in some respects and not others, and which respects it got right is what decides whether a result transfers."

website, training-in-realistic-environments.md

Definition: The adjective is refused as a property of environments and kept as a relation between an environment and a claim. The quotation marks in the page title are the refusal, and the page says so.

Distinguishes it from: A fidelity level, which is a single ordering of environments. This says the ordering does not exist, because the respects can disagree and usually do.

Operational test: Ask which respects. If the answer is a number rather than a list, the word is being used the way this page refuses to use it.

In my project: The last section of the page, and the reason the title keeps punctuation most titles would drop.

the real list and the emulated list

"The substrate, the services, the exploit paths, the five-second polling and the consequences of defender actions are real. The user and attacker populations and the CVE catalogue are emulated."

website, training-in-realistic-environments.md

Definition: Two lists, given in full, with nothing left to inference. Five items real and three emulated, and every claim the project makes has to survive being read against them.

Distinguishes it from: A fidelity claim. This is an inventory, so it can be checked item by item rather than argued about.

Operational test: For any component, would replacing the emulation with the real thing change the defender's observation stream? If yes, it belongs on the real list and is not there yet.

In my project: The same two lists appear on the FOE-Dreamer research page under the heading asking what operational means, and in the manuscript's testbed section. The manuscript's version names the services and is quoted in `cwm_foe_dreamer.tex`.

realistic for what

"Deciding that per claim rather than per environment"

website, training-in-realistic-environments.md

Definition: The rule this page hands to Metrion. Suitability attaches to the pair of an environment and an objective, so the same testbed can be adequate for one claim and inadequate for the next without anything about it changing.

Distinguishes it from: This page's own move, which is to list what is real and let a reader judge. Metrion turns the judging into a procedure with weights and a threshold. The two are consistent and the second is the general form of the first.

Operational test: Name the claim before scoring the environment. If the score was produced first, it is not answering this.

In my project: The link out at the end of the page, and the sentence in the Daedalus README saying Metrion supplies the criteria for deciding which of the three extensions matter for a given claim.

depth over breadth

Definition: The stated choice about where the environment budget went. Eight hosts, each

carrying real services, real exploit code paths and real consequences, rather than a larger topology of thinner hosts.

Distinguishes it from: Scale-first testbeds, which buy host count with per-host fidelity. Neither is a shortfall of the other. They are answering different claims, which is the Metrion point applied to this project's own design.

Operational test: Would adding hosts change what the environment can demonstrate? If the claim is about per-host fidelity, no.

In my project: Section 4.3 of the manuscript states it in those words, and the reviewers split on it. The full entry with both reviewer positions is in `cwm_foe_dreamer.tex`.

1.5 What the budget buys

the number that matters

"The number that matters for this page is the budget, not the loss."

website, training-in-realistic-environments.md

Definition: An instruction about how to read the experiment. The loss result establishes that the method works. The budget result establishes that the route is available to anyone, and only the second is what this page is arguing.

Distinguishes it from: The result as the manuscript reports it, where the loss is the headline because the venue is asking whether the method is better. Same experiment, two numbers, and which one leads depends on which question is being answered.

Operational test: Would the page's argument survive a smaller improvement in loss? Yes. Would it survive a thirty-day budget? No. That asymmetry is what the sentence is naming.

In my project: The experiment section of the page, immediately after the loss and the ablations.

three days on one GPU

"Against both scripted attacker profiles, FOE-Dreamer roughly halves episode loss relative to Rainbow and IQN under matched compute, and trains inside a three-day budget on one GPU."

website, training-in-realistic-environments.md

"Three days on one GPU is what makes training in an emulated network a real option rather than a thought experiment."

website, training-in-realistic-environments.md

Definition: The wall-clock and hardware cost of one training run against the live backend. It is the claim the route turns on, because a route nobody can afford is a position and not a method.

Distinguishes it from: Compute cost in the usual sense. More compute buys more gradient steps on data already collected. This is wall-clock spent waiting for a network to change state, and a bigger machine does not recover it.

Operational test: Divide the budget by the per-step wall-clock and see how many real transitions the run could have contained. If the answer is small relative to what the algorithm class normally

consumes, sample efficiency is carrying the result.

In my project: The experiment section, and the reason the ablations matter: the two components have to be shown to contribute inside that budget rather than asymptotically.

sample efficiency as the binding constraint

"Every step there costs seconds of wall-clock and a real state change on a real host, so what limits the result is sample efficiency, not asymptotic performance."

website, training-in-realistic-environments.md

Definition: The statement that the hard limit on this route is the number of real transitions available, and that a method's asymptotic behaviour is therefore not the thing to compare on.

Distinguishes it from: Sample efficiency as a nice property. Here it is the constraint that selects the method class, and it is why a model-based learner is being used at all.

Operational test: Would one exploratory action have a consequence a user would notice? If yes, the budget is a sample budget and asymptotic curves are answering a different question.

In my project: The manuscript states it as challenge C3, and the entry with the per-interaction costs is in `cwm_foe_dreamer.tex`.

ablations that separate the two components

"Ablations show the factoring and the opponent model each contribute"

website, training-in-realistic-environments.md

Definition: Two removals, each showing that the remaining component does not recover the result on its own. What is being ruled out is that one component is carrying the other while both are credited.

Distinguishes it from: A component sweep, which reports the best combination. This reports what breaks, which is the claim that needs support.

Operational test: Remove one component. Does the result survive? Two removals, two failures, and the credit is joint.

In my project: The page states the conclusion in one line. The tables are in the manuscript.

2 Working

OpenStack

Definition: The cloud platform the network is provisioned on. It supplies the virtual machines, the subnets and the security groups the C2 server and the attacker act through.

Distinguishes it from: A container runtime. Docker is what the sim2sim project's emulation proxy uses, and the difference in substrate is one of the things the two projects do not share.

gRPC

Definition: The transport between the C2 server and the hosts. Each host runs a client, the server holds the connections, and a defender action is a call on one of them.

Distinguishes it from: The action space. gRPC is how the action reaches the machine, not what the actions are. The design page keeps the two separate and so should any description of the testbed.

Cowrie

Definition: The honeypot software the fake-node action stands up. It is what the attacker meets when it reaches a decoy, and it is a running service rather than a modelled one.

SUID

Definition: The Unix set-user-ID permission bit. A binary carrying it runs with its owner's privileges, which is why toggling one is how a false privilege-escalation path gets placed.

Distinguishes it from: A real escalation path, which the attacker could use. The fake edge presents the appearance and not the capability, and an attacker that spends a move on it has spent a move.

the C2 server's vulnerability set

"the C2 server is started with the set of vulnerabilities that host should expose, so the same machine image can present different attack surfaces per experiment"

artifacts/daedalus/use.md

Definition: A per-run argument. The same provisioned image can be brought up with different exposures, which is what makes a second scenario cheap once the network exists.

Distinguishes it from: Reprovisioning. The image does not change. What changes is which services the server is willing to expose on it.

the translation layer

"A translation layer turns the raw network state into the vector the agent reads: per node, its manipulated value, its node state, the three deception flags, and the attacker's perceived state."

artifacts/daedalus/design.md, The observation

Definition: Six fields per node, produced from the live network state. It is where the observation space is defined, and it is deterministic.

Distinguishes it from: An encoder. Nothing here is fitted. The sim2sim project's state alignment step is the same idea done across two environments rather than inside one, and that one does have a fitted stage after it.

the LSTM policy

"The LSTM policy learns where to place deception and when, across far more episodes than a live network could afford."

artifacts/daedalus/use.md

Definition: The recurrent policy trained against the simulator backend in the artifact's own workflow. Placement and timing are the two things it is learning, and the recurrence is there because the attacker's perceived state carries history.

Distinguishes it from: The world-model agent the manuscript reports, which trains against the live backend. The artifact pages document the cheap workflow and the manuscript reports the honest one.

the five-second polling

Definition: The observation cadence. The defender reads the network every five seconds of wall-clock, which is one of the five items on the real list and is what makes a step cost seconds rather than microseconds.

Distinguishes it from: A simulator step, which costs whatever the loop costs. Here the cadence is a property of the deployment and cannot be turned up without making the environment less like the thing it stands for.

3 Peripheral

generated engagements

"so networks and engagements can be generated carrying a known amount of strategic structure rather than fixed by hand"

artifacts/daedalus/README.md, What is being added

The first of the three extensions in progress. It brings the game-generation component of Learn Structure into Daedalus, and the point of it is the phrase known amount: an environment whose strategic content is a parameter can support claims a hand-built one cannot.

adversarial disturbance

"Moving threat modelling and adversarial training inside the environment, where today they are approximated outside it."

artifacts/daedalus/README.md, What is being added

The second extension. It is the same want that the method page states as the open question about adaptive learning adversaries, arriving as an environment change rather than as an algorithm change.

a broader population

"Widening the agent population across attacker, defender and ordinary user, so scenarios vary more and benchmarking means something in practice."

artifacts/daedalus/README.md, What is being added

The third extension. Two fixed attacker profiles and one persona set are what Reviewer B reads as too narrow a basis for a generalization claim, and this is the environment-side answer to that.

the scope statement

"This is research infrastructure for studying deception policies, run against a scripted attacker on infrastructure you provision and control. It is not a tool for operating against systems you do not own."

artifacts/daedalus/use.md, A note on scope

The use page's last section. It is the only sentence in the artifact documentation that is addressed to a reader who might run the thing, and it belongs in the lexicon because it is the boundary the whole artifact is published inside.

4 The clashes, listed

Three, and each is folded into the entry it belongs to above.

Term	The two uses	Where it shows
transfer	Defensive policy moving between two backends of one testbed, here	Offensive policy moving between two simulators, in `td_sim2sim_before_sim2real.tex`
realistic	Scoped per respect, with two lists, here	Refused as posed and replaced by a fit score against an objective, on the Metrion page
environment	The live network, with the word in quotation marks in the design page	The simulator backend, which implements the same interface and is also called the environment

The first is a real ambiguity and the two projects sit next to each other under one heading on the site, which is where a reader will meet it. The second is not a disagreement: this page scopes the adjective by listing respects and Metrion turns the listing into a procedure. The third is the design page's own joke, and it survives because the quotation marks are doing the work.

5 What crosses into other files

The method is not here. The factored world model, the opponent embedding, the exogenous latent, episode loss, matched compute, the operational floor, the baselines and every reviewer position are in `cwm_foe_dreamer.tex`, and the entries above point at it rather than restating it. What this file keeps is the environment and the route.

The realism vocabulary is not here either. Realism dimensions, scoring elements, fit scores, suitability and gain attribution belong to the Metrion file. The two sentences this file needs from it are the ones already quoted: that suitability attaches to a pair, and that the criteria decide which extensions matter for a given claim.

The transfer vocabulary is split on purpose. The sense in which a policy moves between two backends of one interface is defined here, under the two-backend entry. The sense in which a policy moves between two simulators that speak different representations is defined in `td_sim2sim_before_sim2real.tex`. Both are policy transfer in the dictionary's first sense, and `dict_transfer_learning.tex` already records that these two projects use that sense against each other: one removes the step, the other studies it.

6 Sources

- Every quoted sentence is from the website. The locator on each entry says which file and which section. The research page is `content/overview/3-year-agenda/toward-deployment/training-in-realistic-environments.md`.
- The deception framing, the four actions, the three extensions and the Metrion pointer are from `content/artifacts/daedalus/README.md`. The research page does not carry them.
- The C2 mechanism, the action pair, the network fields, the observation layout, the attacker and the two backends are from `content/artifacts/daedalus/design.md`.
- The workflow, the vulnerability-set argument, the LSTM policy, what evaluation reads off and the scope statement are from `content/artifacts/daedalus/use.md`.
- The two lists under what operational means, and the results paragraph, are also on the FOE-Dreamer research page, `cyber-world-modeling/ environment.md`, in the same words.
- The three-project sequence and the question about what the high fidelity training environment will cost are from `content/overview/3-year-agenda/toward-deployment/README.md`.
- The three tiers, the three fluency levels and the four fields are from the Tech Fluency page. The reason a term whose two uses disagree belongs inside its entry is in `lexicon_method.tex`, in this folder.